

COURSE 04 — OBJECT-ORIENTED PROGRAMMING

Dart OOP for Flutter

A practical OOP course in Dart — from classes and constructors to design patterns.

Instructor: Islam Amour Mohamed



14 Modules

Course Roadmap

01 Introduction to OOP

03 Constructors

05 Inheritance

07 Abstract Classes

09 Mixins

11 Object-Oriented Relationships

13 Best Practices

02 Classes and Objects

04 Encapsulation

06 Polymorphism

08 Interfaces in Dart

10 Generics

12 Design Patterns

14 Mini Project — Banking System

MODULE

01

Introduction to OOP

- Why OOP?
- Procedural vs Object-Oriented Programming
- The Four Pillars of OOP

Why OOP?

As programs grow, procedural code — a long list of functions and variables — becomes hard to manage. OOP organizes code around objects: things that have their own data and behavior. This makes code easier to read, reuse, and maintain.

💡 FLUTTER NOTE / ANALOGY

Flutter itself is built entirely with OOP: every widget is an object created from a class.

Procedural vs Object-Oriented Programming

Procedural programming is a sequence of instructions that operate on separate data. OOP groups related data and the functions that work on it together inside a class. This models real-world things more naturally in code.

Dart Example

```
// Procedural style
String name = 'Ali';
int age = 25;
void printStudent(String name, int age) => print('$name, $age');
printStudent(name, age);

// OOP style – data and behavior live together
class Student {
  String name;
  int age;
  Student(this.name, this.age);
  void printInfo() => print('$name, $age');
}
var s = Student('Ali', 25);
s.printInfo();
```

The Four Pillars of OOP

OOP is built on four core principles. Encapsulation: hide internal data, expose only what is needed through a controlled interface. Inheritance: a class can acquire the fields and methods of another class and extend them. Polymorphism: the same method name behaves differently depending on which object runs it. Abstraction: hide complex implementation details and expose only the essential interface to the outside.

MODULE

02

Classes and Objects

- What is a Class?
- What is an Object?
- Fields and Methods
- The this Keyword
- Object References

What is a Class?

A class is a blueprint that defines what data (fields) and behavior (methods) its objects will have. No memory is used when you define a class — it is just a template.

Dart Example

```
class Car {  
  String brand;  
  int speed;  
  
  Car(this.brand, this.speed);  
  
  void accelerate() {  
    speed += 10;  
    print('$brand is now going $speed km/h');  
  }  
}
```

💡 FLUTTER NOTE / ANALOGY

In Flutter, Text, Column, and Scaffold are all classes.

What is an Object?

An object is a concrete instance of a class. When you create one, memory is allocated and fields get real values. You can create many independent objects from the same class, each with its own data.

Dart Example

```
var car1 = Car('Toyota', 0);  
var car2 = Car('BMW', 0);  
  
car1.accelerate(); // Toyota is now going 10 km/h  
car2.accelerate(); // BMW is now going 10 km/h  
// car1 and car2 are completely independent
```

Fields and Methods

Fields are the data a class holds — they describe the state of an object. Methods are functions defined inside a class — they describe what an object can do. Together they make an object both stateful and functional.

Dart Example

```
class BankAccount {  
  String owner; // field  
  double balance; // field  
  
  BankAccount(this.owner, this.balance);  
  
  void deposit(double amount) { // method  
    balance += amount;  
    print('New balance: $balance');  
  }  
  
  void withdraw(double amount) { // method  
    balance -= amount;  
    print('Remaining: $balance');  
  }  
}
```

The this Keyword

'this' refers to the current object — the instance that is running the method. It is most commonly used to resolve the conflict between a field name and a parameter with the same name.

Dart Example

```
class Student {  
  String name;  
  
  // Without shorthand: 'this.name' is the field, 'name'  
  // is the parameter  
  Student(String name) {  
    this.name = name;  
  }  
  
  void introduce() => print('Hi, I am ${this.name}');  
}
```

Object References

In Dart, variables do not store objects directly — they store a reference (a pointer) to where the object lives in memory. If you assign one object variable to another, both point to the same object. Changing one will affect the other.

Dart Example

```
var a = BankAccount('Ali', 1000);  
var b = a; // b points to the SAME object as a  
  
b.deposit(500);  
print(a.balance); // 1500 — a is also affected because  
they share the same object
```

MODULE

03

Constructors

- Default and Parameterized Constructors
- Named Constructors
- Constant Constructors
- Initializer List
- Redirecting Constructors
- Constructor Execution Order

Default and Parameterized Constructors

A constructor is a special method that runs automatically when an object is created. If you do not define one, Dart provides a default no-argument constructor. A parameterized constructor lets you pass values at creation time. Dart also supports a shorthand syntax — 'this.fieldName' in the parameter list — which assigns the argument directly to the field without writing a body.

Dart Example

```
class Person {  
  String name;  
  int age;  
  
  // Shorthand parameterized constructor  
  Person(this.name, this.age);  
}  
  
var p = Person('Ali', 25);  
print(p.name); // Ali  
print(p.age);  // 25
```

Named Constructors

Dart allows multiple constructors on the same class by giving them names. This is useful when you need different ways to create an object — for example, a guest user with default values, or an object built from a Map.

Dart Example

```
class Person {  
  String name;  
  int age;  
  
  Person(this.name, this.age);  
  
  // Named constructor with defaults  
  Person.guest() : name = 'Guest', age = 0;  
  
  // Named constructor from a Map (e.g. JSON)  
  Person.fromMap(Map<String, dynamic> map)  
    : name = map['name'],  
      age = map['age'];  
}  
  
var guest = Person.guest();  
var fromData = Person.fromMap({'name': 'Sara', 'age':  
22});  
print(guest.name);    // Guest  
print(fromData.name); // Sara
```

💡 FLUTTER NOTE / ANALOGY

Flutter uses this constantly: `EdgeInsets.all()`, `EdgeInsets.symmetric()`, and `EdgeInsets.only()` are all named

Constant Constructors

A const constructor creates compile-time constant objects. All fields must be final. When two identical const objects are created, Dart reuses the same instance in memory instead of allocating a new one.

Dart Example

```
class Point {  
  final int x;  
  final int y;  
  
  const Point(this.x, this.y);  
}  
  
const p1 = Point(0, 0);  
const p2 = Point(0, 0);  
print(identical(p1, p2)); // true – same object in memory,  
not two copies
```

💡 FLUTTER NOTE / ANALOGY

This is how const Text('Hello') works in Flutter — Flutter reuses the same widget object across rebuilds, skipping unnecessary

Initializer List

An initializer list runs before the constructor body using a colon ':' after the parameter list. It is used to set final fields that require a computation, or to call the parent class constructor with 'super'.

Dart Example

```
class Rectangle {  
    final double width;  
    final double height;  
    final double area;  
  
    Rectangle(this.width, this.height) : area = width *  
    height;  
    // 'area' is computed and set before the body runs  
}  
  
var r = Rectangle(4, 5);  
print(r.area); // 20.0
```

Redirecting Constructors

A redirecting constructor delegates to another constructor in the same class using 'this(...)'. It has no body. This is useful for providing shortcuts with preset values.

Dart Example

```
class Point {  
  double x;  
  double y;  
  
  Point(this.x, this.y);  
  
  // Redirects to Point(0, 0)  
  Point.origin() : this(0, 0);  
}  
  
var p = Point.origin();  
print('${p.x}, ${p.y}'); // 0.0, 0.0
```

Constructor Execution Order

When a child class object is created, the execution order is: (1) the parent's initializer list runs, (2) the parent's constructor body runs, (3) the child's initializer list runs, (4) the child's constructor body runs. Understanding this order matters when fields depend on each other across the hierarchy.

Dart Example

```
class Animal {  
  String name;  
  Animal(this.name) {  
    print('Animal constructor: $name');  
  }  
}  
  
class Dog extends Animal {  
  String breed;  
  Dog(String name, this.breed) : super(name) {  
    print('Dog constructor: $breed');  
  }  
}  
  
var d = Dog('Rex', 'Labrador');  
// Animal constructor: Rex  
// Dog constructor: Labrador
```

MODULE

04

Encapsulation

- Public vs Private Members
- Getters
- Setters with Validation

Public vs Private Members

In Dart, any field or method whose name starts with an underscore '_' is private to its file (library). It cannot be accessed from outside that file. Everything else is public by default. This prevents external code from putting the object into an invalid state.

Dart Example

```
class BankAccount {  
  String owner;  
  double _balance; // private – not accessible outside  
  this file  
  
  BankAccount(this.owner, this._balance);  
  
  // Controlled read-only access  
  double get balance => _balance;  
  
  void deposit(double amount) {  
    if (amount > 0) _balance += amount;  
  }  
}  
  
var acc = BankAccount('Ali', 1000);  
print(acc.balance); // 1000 – OK, uses getter  
// acc._balance = -500; // ERROR if accessed from another  
file
```

💡 FLUTTER NOTE / ANALOGY

In Flutter, widget state variables are typically private — for example, `_counter` or `_isLoading`.

Getters

A getter is a method that looks like a property when accessed. It lets you compute or control the value returned when someone reads a field, without exposing the raw field directly.

Dart Example

```
class Circle {  
  final double radius;  
  Circle(this.radius);  
  
  double get area => 3.14 * radius * radius;  
  double get diameter => radius * 2;  
}  
  
var c = Circle(5);  
print(c.area);      // 78.5  
print(c.diameter);  // 10.0  
// area and diameter look like fields but are computed on  
demand
```

Setters with Validation

A setter is a method that runs when a value is assigned to a property. It lets you validate or transform the incoming data before storing it — preventing invalid state from entering the object.

Dart Example

```
class Student {  
  String name;  
  double _grade = 0;  
  
  Student(this.name);  
  
  double get grade => _grade;  
  
  set grade(double value) {  
    if (value >= 0 && value <= 100) {  
      _grade = value;  
    } else {  
      print('Invalid grade: $value. Must be between 0 and  
100.');    }  
  }  
}  
  
var s = Student('Ali');  
s.grade = 85;    // OK  
s.grade = 150;  // Invalid grade: 150. Must be between 0  
and 100.  
print(s.grade); // 85
```

MODULE

05

Inheritance

- Why Inheritance?
- The super Keyword
- Method Overriding and @override

Why Inheritance?

Inheritance allows a class (child) to reuse the fields and methods of another class (parent) using the 'extends' keyword. Instead of rewriting shared logic, you define it once in the parent and let children inherit it. A child can also add new behavior on top.

Dart Example

```
class Animal {
  String name;
  Animal(this.name);

  void eat() => print('$name is eating');
  void sleep() => print('$name is sleeping');
}

class Dog extends Animal {
  Dog(String name) : super(name);

  void bark() => print('$name says: Woof!');
}

var dog = Dog('Rex');
dog.eat();    // Rex is eating – inherited from Animal
dog.bark();   // Rex says: Woof! – Dog's own method
```

💡 FLUTTER NOTE / ANALOGY

In Flutter, every widget you write extends either StatelessWidget or StatefulWidget to inherit Flutter's widget

The super Keyword

'super' refers to the parent class. You use it in two ways: in the initializer list to call the parent's constructor, and inside an overridden method to call the parent's version of that method.

Dart Example

```
class Animal {
  String name;
  Animal(this.name);
  void describe() => print('I am an animal named $name');
}

class Cat extends Animal {
  String color;

  Cat(String name, this.color) : super(name); // calls
  Animal's constructor

  @override
  void describe() {
    super.describe();           // calls Animal's
  describe()
    print('I am a $color cat'); // then adds its own
  line
  }
}

Cat('Luna', 'white').describe();
// I am an animal named Luna
// I am a white cat
```

Method Overriding and @override

A child class can replace a parent method with its own version. The `@override` annotation is not required but strongly recommended — it tells Dart (and the reader) that this is intentionally replacing the parent's method.

Dart Example

```
class Shape {  
  double area() => 0;  
}  
  
class Circle extends Shape {  
  double radius;  
  Circle(this.radius);  
  
  @override  
  double area() => 3.14 * radius * radius;  
}  
  
class Rectangle extends Shape {  
  double width, height;  
  Rectangle(this.width, this.height);  
  
  @override  
  double area() => width * height;  
}  
  
print(Circle(5).area());           // 78.5  
print(Rectangle(4, 6).area());    // 24.0
```

💡 FLUTTER NOTE / ANALOGY

Every Flutter widget overrides the `build()` method — that is the core contract.

MODULE

06

Polymorphism

- Same Interface, Different Behavior
- Parent References

Same Interface, Different Behavior

Polymorphism means 'many forms'. When multiple classes share the same method name but each implements it differently, you can call the method without caring about the exact type. Dart decides at runtime which version to call — this is called dynamic dispatch.

Dart Example

```
class Animal {  
  void makeSound() => print('...');  
}  
  
class Dog extends Animal {  
  @override  
  void makeSound() => print('Woof!');  
}  
  
class Cat extends Animal {  
  @override  
  void makeSound() => print('Meow!');  
}  
  
List<Animal> animals = [Dog(), Cat(), Dog()];  
for (var a in animals) {  
  a.makeSound(); // Woof! Meow! Woof!  
  // Dart picks the correct version at runtime for each  
  object  
}
```

Parent References

You can store a child object in a parent-typed variable. The variable only knows the parent's interface, but at runtime, Dart calls the child's overridden method. This is powerful because your code stays generic while behavior stays specific.

Dart Example

```
Animal animal = Dog(); // Dog object stored in an Animal
                           variable
animal.makeSound();      // Woof! – runs Dog's version, not
                           Animal's

// Useful for functions that work with any Animal
void describeSound(Animal a) => a.makeSound();

describeSound(Dog()); // Woof!
describeSound(Cat()); // Meow!
```

MODULE

07

Abstract Classes



What is an Abstract Class?

What is an Abstract Class?

An abstract class defines a contract — it can declare methods without implementing them (abstract methods). Any class that extends it must provide an implementation for every abstract method. An abstract class cannot be instantiated directly; it only exists to be extended.

💡 FLUTTER NOTE / ANALOGY

StatelessWidget in Flutter is abstract — you cannot use it directly, you must extend it and implement build().

Dart Example

```
abstract class Shape {
  double area();      // abstract – no body, must be
                      // implemented
  double perimeter(); // abstract – no body, must be
                      // implemented

  void describe() {   // concrete method – has a body,
    // inherited as-is
    print('Area: ${area()}, Perimeter: ${perimeter()}');
  }
}
```

```
class Circle extends Shape {
  double radius;
  Circle(this.radius);

  @override
  double area() => 3.14 * radius * radius;

  @override
  double perimeter() => 2 * 3.14 * radius;
}
```

```
// Shape s = Shape(); // ERROR: cannot instantiate an
// abstract class
```

```
Shape s = Circle(5);
```

```
print(s.area()); // 78.5 print(s.perimeter()); // 31.4
```


MODULE

08

Interfaces in Dart

- Every Class is an Interface
- extends vs implements
- Multiple Interfaces

Every Class is an Interface

Dart does not have a separate 'interface' keyword. Every class automatically defines an interface — the set of public methods and fields it exposes. Another class can use that interface with 'implements'. When you implement a class, you must provide your own version of every method. You inherit the contract but none of the implementation.

Dart Example

```
class Animal {  
  void makeSound() => print('...');  
  void eat() => print('eating');  
}  
  
// Bird implements Animal — must define ALL methods itself  
class Bird implements Animal {  
  @override  
  void makeSound() => print('Tweet!');  
  
  @override  
  void eat() => print('Bird is eating seeds');  
}
```

extends vs implements

'extends' gives you the parent's full code — fields, methods, and their implementations. You only override what you want to change. 'implements' gives you only the contract — you must write every method yourself from scratch. Use extends when classes share real behavior. Use implements when you only want to guarantee a class fulfills a certain interface.

Dart Example

```
// extends: inherits implementation, override only what changes
class Dog extends Animal {
  // eat() and makeSound() are inherited automatically
  @override
  void makeSound() => print('Woof!');
}

// implements: gets nothing, must implement everything manually
class Robot implements Animal {
  @override
  void makeSound() => print('Beep boop');

  @override
  void eat() => print('Robots do not eat');
}
```

Multiple Interfaces

A class can implement multiple interfaces at the same time. Dart only allows single inheritance (one extends), but multiple implements. This lets a class fulfill several contracts at once.

Dart Example

```
class Flyable {  
  void fly() {}  
}  
  
class Swimmable {  
  void swim() {}  
}  
  
class Duck implements Flyable, Swimmable {  
  @override  
  void fly() => print('Duck is flying');  
  
  @override  
  void swim() => print('Duck is swimming');  
}  
  
Duck d = Duck();  
d.fly(); // Duck is flying  
d.swim(); // Duck is swimming
```

MODULE

09

Mixins

- Why Mixins?
- Mixin Restrictions and the on Keyword

Why Mixins?

Mixins solve a problem inheritance alone cannot: sharing behavior across unrelated classes without forcing them into the same hierarchy. A mixin is a reusable bundle of methods you can add to any class using 'with'. Unlike interfaces, mixins come with actual implementations — you get the code, not just the contract.

Dart Example

```
mixin Flyable {  
  void fly() => print('$runtimeType is flying');  
}  
  
mixin Swimmable {  
  void swim() => print('$runtimeType is swimming');  
}  
  
class Bird with Flyable {}  
class Fish with Swimmable {}  
class Duck with Flyable, Swimmable {} // gets both  
behaviors  
  
Duck().fly(); // Duck is flying  
Duck().swim(); // Duck is swimming
```

💡 FLUTTER NOTE / ANALOGY

Flutter's `TickerProviderStateMixin` and `AutomaticKeepAliveClientMixin` are real examples of mixins

Mixin Restrictions and the on Keyword

By default, a mixin can be applied to any class. The 'on' keyword restricts a mixin so it can only be used on a specific class or its subclasses. This lets the mixin safely access fields and methods of that class.

Dart Example

```
class Animal {
  String name = 'Unknown';
}

mixin Logger on Animal {
  // Can safely access 'name' because this mixin is
  // restricted to Animal
  void log(String action) => print('[LOG] $name:
$action');
}

class Dog extends Animal with Logger {}

var dog = Dog();
dog.name = 'Rex';
dog.log('fetched the ball'); // [LOG] Rex: fetched the
ball
```

MODULE

10

Generics

- Why Generics?
- Generic Methods
- Multiple Generic Types
- Generic Constraints

Why Generics?

Without generics, you would need a separate class for every data type — a `BoxForInt`, a `BoxForString`, and so on. Generics let you write one class or function that works with any type while remaining fully type-safe. The type is specified when you use the class.

Dart Example

```
// Without generics – loses type safety
class Box {
  dynamic value;
  Box(this.value);
}

// With generics – type-safe and reusable
class Box<T> {
  T value;
  Box(this.value);
}

var intBox = Box<int>(42);
var strBox = Box<String>('Hello');
print(intBox.value); // 42
print(strBox.value); // Hello
// Box<int> won't accept a String – caught at compile time
```

💡 FLUTTER NOTE / ANALOGY

`List<T>`, `Map<K,V>`, `Future<T>`, and `StreamBuilder<T>` in Flutter are all generic.

Generic Methods

A function can also be generic. The type parameter is declared on the function and determined at the call site. This lets you write reusable utility functions that work on any type.

Dart Example

```
T getFirst<T>(List<T> list) {  
  if (list.isEmpty) throw Exception('List is empty');  
  return list.first;  
}  
  
print(getFirst<int>([10, 20, 30]));           // 10  
print(getFirst<String>(['a', 'b', 'c']));    // a
```

Multiple Generic Types

A class can have more than one type parameter. The built-in `Map<K, V>` works exactly this way — `K` is the key type and `V` is the value type.

Dart Example

```
class Pair<K, V> {  
  K key;  
  V value;  
  Pair(this.key, this.value);  
  
  @override  
  String toString() => '$key: $value';  
}  
  
var p1 = Pair<String, int>('score', 95);  
var p2 = Pair<String, bool>('isAdmin', true);  
print(p1); // score: 95  
print(p2); // isAdmin: true
```

Generic Constraints

You can restrict a generic type using 'extends'. This ensures the type parameter is a specific class or its subclass, allowing you to safely call methods defined on that class inside the generic code.

Dart Example

```
class Repository<T extends Object> {  
  final List<T> _items = [];  
  
  void add(T item) => _items.add(item);  
  T get(int index) => _items[index];  
  List<T> getAll() => List.unmodifiable(_items);  
  int get count => _items.length;  
}  
  
var repo = Repository<String>();  
repo.add('Flutter');  
repo.add('Dart');  
print(repo.getAll()); // [Flutter, Dart]  
print(repo.count);    // 2
```

MODULE

11

Object-Oriented Relationships

- Association
- Aggregation
- Composition

Association

Association is the loosest relationship: two objects know about each other but neither owns the other. Both exist independently and can be linked or unlinked at any time.

Dart Example

```
class Teacher {
  String name;
  Teacher(this.name);
}

class Student {
  String name;
  Teacher? teacher; // knows about a Teacher but doesn't
  own one
  Student(this.name);
}

var t = Teacher('Mr. Smith');
var s = Student('Ali');
s.teacher = t; // linked
s.teacher = null; // unlinked – both still exist
independently
```

Aggregation

Aggregation is a 'has-a' relationship where one object contains references to others, but those objects can exist independently. Destroying the container does not destroy the contained objects.

Dart Example

```
class Student {  
    String name;  
    Student(this.name);  
}  
  
class Classroom {  
    String roomNumber;  
    List<Student> students; // aggregation – students exist  
    outside Classroom  
    Classroom(this.roomNumber, this.students);  
}  
  
var s1 = Student('Ali');  
var s2 = Student('Sara');  
var room = Classroom('101', [s1, s2]);  
// s1 and s2 still exist if room is discarded
```

Composition

Composition is a stronger 'has-a' relationship. The contained object is created by and belongs to the container — it cannot exist without it.

💡 FLUTTER NOTE / ANALOGY

Flutter uses composition heavily: a Column is composed of its children, a Card is composed of its child. This 'composition over

Dart Example

```
class Engine {  
  int horsepower;  
  Engine(this.horsepower);  
  void start() => print('Engine started ($horsepower  
HP)');  
}  
  
class Car {  
  String brand;  
  Engine engine; // Engine is created by Car – belongs to  
  Car  
  
  Car(this.brand, int hp) : engine = Engine(hp);  
  
  void drive() {  
    engine.start();  
    print('$brand is driving');  
  }  
}  
  
Car('Toyota', 150).drive();  
// Engine started (150 HP)  
// Toyota is driving
```


MODULE

12

Design Patterns

- Singleton Pattern
- Factory Pattern
- Observer Pattern

Singleton Pattern

The Singleton pattern ensures that only one instance of a class exists throughout the entire app. It uses a private constructor so no one can call 'new' from outside, a static field that holds the single instance, and a factory constructor that always returns that same instance.

💡 FLUTTER NOTE / ANALOGY

Common use cases in Flutter: a shared HTTP client, a database connection, or an app-wide logger.

Dart Example

```
class AppLogger {
  static final AppLogger _instance =
    AppLogger._internal();

  AppLogger._internal(); // private constructor

  factory AppLogger() => _instance; // always returns the
  same object

  void log(String message) => print('[LOG] $message');
}

void main() {
  var logger1 = AppLogger();
  var logger2 = AppLogger();

  print(identical(logger1, logger2)); // true – same
  object
  logger1.log('App started');          // [LOG] App started
}
```

Factory Pattern

The Factory pattern uses a factory constructor to control object creation. Instead of always creating a new object, the factory can decide which concrete subclass to return based on input. The caller only knows the abstract type — not which specific class was created.

Dart Example

```
abstract class Shape {
  double area();

  factory Shape.create(String type, double size) {
    if (type == 'circle') return Circle(size);
    if (type == 'square') return Square(size);
    throw ArgumentError('Unknown shape type: $type');
  }
}

class Circle extends Shape {
  double radius;
  Circle(this.radius);
  @override
  double area() => 3.14 * radius * radius;
}

class Square extends Shape {
  double side;
  Square(this.side);
  @override
  double area() => side * side;
}

Shape s1 = Shape.create('circle', 5);
Shape s2 = Shape.create('square', 4);
print('1: ${s1.area()} 2: ${s2.area()}');
```

Observer Pattern

The Observer pattern defines a one-to-many relationship: when one object (the subject) changes state, all subscribed observers are automatically notified. This is the foundation of reactive programming. In Dart, Stream and StreamController are a direct implementation of this pattern.

💡 FLUTTER NOTE / ANALOGY

BLoC — the primary Flutter state management approach — is built entirely on this idea: the BLoC is the subject, widgets are

Dart Example

```
import 'dart:async';

class WeatherStation {
  final _controller =
    StreamController<double>.broadcast();

  Stream<double> get updates => _controller.stream;

  void setTemperature(double temp) {
    print('Station broadcast: $temp°C');
    _controller.add(temp); // notifies all subscribers
  }

  void dispose() => _controller.close();
}

void main() {
  var station = WeatherStation();

  station.updates.listen((t) => print('Phone: $t°C'));
  station.updates.listen((t) => print('TV: $t°C'));

  station.setTemperature(22.5);
  // Station broadcast: 22.5°C
  // Phone: 22.5°C
  // TV: 22.5°C
```

MODULE

13

Best Practices

- Composition Over Inheritance
- Favor Immutability
- Single Responsibility Principle
- Prefer Private Fields and Meaningful Names

Composition Over Inheritance

Prefer building complex behavior by combining small focused objects rather than creating deep inheritance chains. Deep inheritance is fragile — a change in the parent can break all children. Composition is more flexible and easier to modify independently.

Dart Example

```
// Fragile deep inheritance
// Animal -> Pet -> Dog -> TrainedDog -> CompetitionDog

// Better: compose behavior
class Dog {
  final Trainer? trainer; // optional composed object
  final String name;
  Dog(this.name, {this.trainer});

  void perform() => trainer != null
    ? print('$name performs a trick')
    : print('$name sits quietly');
}
```

💡 FLUTTER NOTE / ANALOGY

Flutter's widget system is built on this principle.

Favor Immutability

Make fields final whenever possible. Immutable objects cannot be changed after creation, which eliminates a whole class of bugs where state is accidentally modified.

Dart Example

```
// Mutable – risky
class Point {
  double x, y;
  Point(this.x, this.y);
}

// Immutable – safer, supports const
class Point {
  final double x;
  final double y;
  const Point(this.x, this.y);
}
```

💡 FLUTTER NOTE / ANALOGY

Flutter widget fields must be final — all state changes happen in the State object, not the widget.

Single Responsibility Principle

Each class should have one clear job — one reason to change. A class that fetches data, validates it, and renders it is doing too much and becomes hard to test and maintain.

Dart Example

```
// Bad: one class does everything
class UserManager {
  void fetchFromApi() {}
  void validateEmail(String email) {}
  void renderOnScreen() {}
}

// Good: each class has one job
class UserRepository { void fetch() {} }
class UserValidator { bool validate(String email) =>
  email.contains('@'); }
// UI is handled by a widget separately
```

💡 FLUTTER NOTE / ANALOGY

In Flutter, keep UI (widgets), logic (BLoC/controller), and data (repository/service) in separate classes.

Prefer Private Fields and Meaningful Names

Make fields private by default and expose them through getters only when needed. Also use names that clearly communicate intent. Avoid single-letter names outside of short loop counters. Clear names make code self-documenting.

Dart Example

```
// Hard to read
class A {
  int x;
  String s;
  A(this.x, this.s);
}

// Clear and safe
class Student {
  final String _name;
  int _grade;

  Student(this._name, this._grade);

  String get name => _name;
  int get grade => _grade;
}
```

MODULE

14

Mini Project — Banking System



Project Overview

Project Overview

Build a small banking system that applies every OOP concept from the course: a Singleton Bank, an abstract Account base class, concrete SavingsAccount and CurrentAccount subclasses, a Logger mixin for transaction logging, a generic Repository<T> for storage, and the Observer pattern via StreamController for live transaction notifications.

Dart Example

```
import 'dart:async';

// --- Logger Mixin ---
mixin Logger {
  void log(String msg) => print('[LOG] $msg');
}

// --- Generic Repository ---
class Repository<T> {
  final List<T> _items = [];
  void add(T item) => _items.add(item);
  List<T> getAll() => List.unmodifiable(_items);
}

// --- Abstract Account ---
abstract class Account with Logger {
  final String owner;
  double _balance;
  final _controller =
    StreamController<String>.broadcast();

  Account(this.owner, this._balance);

  double get balance => _balance;
  Stream<String> get transactions => _controller.stream;
}
```